Patient Module
Frontend Integration & Production Guide

AbrenCare Medical Application
Updated Medical Document Versioning Architecture
Version 2.0 — August 2026

1. Purpose and Scope

- This document is the updated frontend integration and production guide for the Patient module.
- The module uses accounts.User as the source of truth for identity and account information. Patient stores patient-domain information, while medical documents and clinical records have separate responsibilities.

2. Architecture and Responsibilities

- User: identity, authentication, account status, contact/address information, and profile information.
- Patient: patient-specific domain profile linked one-to-one with User.
- EmergencyContact: emergency contacts belonging to a Patient.
- MedicalRecord: official clinical information and history; it is not a patient-editable document store.
- MedicalDocument: files uploaded by or on behalf of a Patient. Documents are versioned rather than silently overwritten.

3. User as the Source of Truth

- Patient must not duplicate authoritative User fields such as full name, email, phone number, address, city, postal code, date of birth, or profile picture.
- Frontend applications should obtain these values from the Patient response as exposed by the backend from the related User.
- The frontend must never supply patient ownership or uploaded_by values for /me/ operations.

4. Patient Lifecycle

- An authenticated User may create one Patient profile. Patient onboarding does not require the Doctor-style administrative approval workflow.
- Account deactivation operates on User.is_active.
- Deactivation preserves the Patient profile, medical history, and document relationships.
- Patient profile deletion should not be exposed as a normal account operation.

5. Patient Profile API

- GET /api/patients/me/ — retrieve the authenticated patient's profile.
- POST /api/patients/me/ — create the authenticated user's Patient profile.
- PATCH /api/patients/me/ — update patient-specific fields.
- GET /api/patients/<id>/ — retrieve a patient only when authorization permits.
- PATCH /api/patients/<id>/ — update only when authorization permits.
- DELETE /api/patients/<id>/ — unsupported; return 405.

6. Account Deactivation

- POST /api/patients/me/deactivate/ — deactivates the authenticated User account.
- The Patient object and related medical data remain stored.
- After a successful response, the frontend should clear tokens/session state and return to the signed-out flow.

7. Emergency Contacts

- GET /api/patients/me/emergency-contacts/ — list the current patient's contacts.
- POST /api/patients/me/emergency-contacts/ — create an owned contact.
- GET /api/patients/me/emergency-contacts/<id>/ — retrieve an owned contact.
- PATCH /api/patients/me/emergency-contacts/<id>/ — update an owned contact.
- DELETE /api/patients/me/emergency-contacts/<id>/ — delete an owned contact.
- Ownership is derived from request.user.

8. Medical Documents — Updated Concept

- A MedicalDocument is a patient-owned supporting file, such as a laboratory report, prescription, imaging report, referral letter, or previous medical report.
- A patient may upload their own medical documents.
- A patient may view their own document metadata/file through authorized endpoints.
- A patient should not edit the contents of an existing document.
- To correct a document, use the replacement/versioning workflow rather than silently overwriting the old file.
- An uploaded MedicalDocument does not automatically become a MedicalRecord.

9. Updated MedicalDocument Model

- Fields: patient, uploaded_by, file, description, version, is_current, replaced_document, and uploaded_at.
- patient uses a protected relationship so document ownership is not casually removed with a Patient deletion.
- uploaded_by records who uploaded the file.
- version starts at 1 and increments when a document is replaced.
- is_current identifies the latest version.
- replaced_document links a new version to the document it replaced.
- uploaded_at records when that version was uploaded.
- Recommended indexes support patient history and current-document queries.

10. Medical Document Versioning Workflow

- Initial upload: create version 1 with is_current=True.
- Replacement: mark the previous current version is_current=False, create a new document with version + 1, set replaced_document to the previous version, and mark the new document current.
- Example: Blood_Test.pdf v1 → Blood_Test.pdf v2 → Blood_Test.pdf v3.
- Previous versions should not be silently overwritten.
- A uniqueness rule should prevent one document from being replaced by multiple successor documents.

11. Medical Document API

- GET /api/patients/me/medical-documents/ — list the patient's documents.
- POST /api/patients/me/medical-documents/ — upload a new document.
- GET /api/patients/me/medical-documents/<id>/ — retrieve an owned document.
- DELETE /api/patients/me/medical-documents/<id>/ — delete according to the module's retention policy.
- POST /api/patients/me/medical-documents/<id>/replace/ — upload a replacement version for the current owned document.

12. Medical Document Upload Rules

- The current contract permits PDF, JPG, JPEG, and PNG files.
- The current size limit is 10 MB.
- The backend must validate file type and size; frontend validation is only for user experience.
- For production, add MIME/content validation, private object storage, malware scanning, controlled download URLs, and appropriate retention policies.

13. Medical Records

- MedicalRecord represents official clinical information such as diagnosis and treatment.
- Patients may view their own records when authorized.
- Patients should not create, update, or delete official clinical records through the patient-facing API.
- Doctor/clinical workflows should be responsible for creating or updating clinical information.
- MedicalDocument may provide supporting evidence for a clinical decision, but it does not automatically create or modify MedicalRecord.

14. Authentication and Authorization

- Protected patient endpoints require authentication.
- Ownership must be enforced server-side on every patient-owned object.
- /me/ routes are preferred because they eliminate unnecessary patient-ID input from the frontend.
- Never trust a client-supplied patient_id or uploaded_by field to establish ownership.
- Do not expose unrestricted patient data to ordinary patients.
- Future Doctor access should be based on an explicit clinical/business relationship such as an appointment or care relationship.

15. Frontend Integration and Error Handling

- 401 Unauthorized: authentication/session problem.
- 403 Forbidden: authorization failure or inactive account.
- 404 Not Found: resource does not exist within the permitted scope.
- 405 Method Not Allowed: operation intentionally unsupported, such as deleting the Patient profile.
- 409 Conflict: Patient profile already exists or another defined uniqueness conflict occurs.
- 400 Bad Request: validation failure.
- Use multipart/form-data for medical-document uploads and replacements.

16. Example Frontend Flows

- Profile setup: login → GET /patients/me/ → if 404, show patient profile creation → POST /patients/me/.
- Document upload: select file → client-side size/type check → POST /patients/me/medical-documents/ → refresh document list.
- Document replacement: choose current document → select corrected file → POST /patients/me/medical-documents/{id}/replace/ → display new version.
- Account deactivation: confirm action → POST /patients/me/deactivate/ → clear local authentication state → redirect to login.

17. Security and Privacy Recommendations

- Medical documents should use private storage rather than publicly accessible media URLs.
- Prefer short-lived signed download URLs or an authorized download endpoint.
- Add audit logging for document upload, access, replacement, and deletion.
- Use rate limiting for uploads and sensitive endpoints.
- Consider malware scanning and content validation before making uploaded files available.

- Define a retention policy before enabling permanent deletion in production.

18. Recommended Database Rules

- Patient.user: OneToOneField(User, on_delete=PROTECT, related_name='patient_profile').
- MedicalDocument.patient: ForeignKey(Patient, on_delete=PROTECT, related_name='medical_documents').
- MedicalDocument.uploaded_by: ForeignKey(User, on_delete=PROTECT, related_name='uploaded_medical_documents').
- Consider a conditional unique constraint on replaced_document so each version has at most one direct successor.
- Consider a conditional constraint ensuring only one current version exists for a logical document chain.

19. Testing Checklist

- User can create only one Patient profile.
- Patient ownership cannot be changed through request data.
- Identity fields remain controlled by User.
- Patient deletion is not exposed.
- Deactivation preserves Patient and medical data.
- Patient can upload an allowed document within 10 MB.
- Invalid extensions and oversized files are rejected.
- Patient can view only their own documents.
- Patient can replace a current document and receives version 2, version 3, etc.
- Previous document versions become non-current and are not overwritten.
- Patient cannot modify MedicalRecord through patient-facing endpoints.
- One document cannot have multiple successor replacements.
- Unauthorized users cannot access another patient's documents.

20. Appointment Module Integration

- Appointment should directly relate Patient and Doctor.
- Appointment will provide an important authorization boundary for future clinical access.
- A Doctor should not automatically receive access to every Patient or every MedicalDocument.
- Access can later be granted based on appointment status, consultation, care relationship, or another explicit permission rule.
- The Patient module should remain stable while Appointment becomes the bridge between patient and doctor workflows.

21. Final Architecture

- User → identity and account source of truth.
- Patient → patient-domain profile.
- Patient → EmergencyContact.
- Patient → MedicalDocument → version 1, version 2, version 3, etc.
- Patient → MedicalRecord → official clinical history, controlled by clinical workflows.
- Appointment → future bridge connecting Patient and Doctor for authorized clinical interaction.

22. Endpoint Reference Table

Method	Endpoint	Purpose
GET	/api/patients/me/	Current patient profile
POST	/api/patients/me/	Create patient profile

Method	Endpoint	Purpose
PATCH	/api/patients/me/	Update patient fields
POST	/api/patients/me/deactivate/	Deactivate account
GET	/api/patients/me/emergency-contacts/	List contacts
POST	/api/patients/me/emergency-contacts/	Create contact
PATCH	/api/patients/me/emergency-contacts/{id}/	Update contact
DELETE	/api/patients/me/emergency-contacts/{id}/	Delete contact
GET	/api/patients/me/medical-records/	List clinical records
GET	/api/patients/me/medical-records/{id}/	View clinical record
GET	/api/patients/me/medical-documents/	List documents
POST	/api/patients/me/medical-documents/	Upload document
GET	/api/patients/me/medical-documents/{id}/	View document
DELETE	/api/patients/me/medical-documents/{id}/	Delete per retention policy
POST	/api/patients/me/medical-documents/{id}/replace/	Create replacement version

Final design principle: The User model remains the source of truth for identity and account state. Patients own their uploaded supporting documents, while official clinical records remain controlled by authorized clinical workflows. Document replacement creates a new version instead of overwriting history.